

Terraform – Interview Questions & Answers

what is terraform init?

Terraform init initializes the Terraform project. It downloads required providers and modules and configures the backend for storing the state.

What is Terraform drift?

Terraform drift = When the real infrastructure is different from what Terraform expects.

How would you manage one Terraform codebase across multiple environments?

I keep one Terraform codebase with reusable modules, and a separate folder for each environment: dev, staging, and prod. Each environment has its own variables and its own remote state, for example an S3 bucket with DynamoDB locking, so one environment can never affect another.

I don't share state across environments. Workspaces can also give separate state, but for production I prefer separate directories and backends. That isolation lowers the risk of applying changes to the wrong environment by mistake.

For production, changes go through CI/CD with a plan review and a manual approval before apply."

What is Terraform state locking?

Terraform State Locking

1. What is it?

Terraform state locking prevents multiple users/processes from modifying the same Terraform state at the same time.

2. Why is it needed?

- Terraform stores infrastructure information in a state file.
- If two people run terraform apply simultaneously, they could modify the state at the same time.
- This can cause state corruption or inconsistent infrastructure.

3. How does it work?

- User A starts terraform apply.
- Terraform acquires a state lock.
- User B tries terraform apply.
- Terraform sees the state is locked and waits/errors instead of modifying it.
- After User A finishes, the lock is released.

How would you resolve a Terraform state-locking issue?

```
terraform plan
```

If you see:

Error acquiring the state lock

The state is currently locked.

2. Check if another Terraform operation is running

- Check with your team.
- Check Jenkins/GitHub Actions/GitLab pipelines.
- Check whether another terraform apply is running.

3. Wait if another operation is running

- Do not force-unlock while another Terraform operation is active.
- Let it complete and release the lock.

4. Identify the Lock ID

- Terraform normally shows a Lock ID in the error message.

Example:

ID: 12345678-xxxx-xxxx

If it is a stale lock

If you confirm that no Terraform operation is running:

```
terraform force-unlock <LOCK_ID>
```

Run Terraform again

```
terraform plan
```

Then:

```
terraform apply
```

Prevent the issue

- Use a remote backend.
- Enable state locking.
- Avoid multiple CI/CD jobs applying the same environment simultaneously.
- Use CI/CD approval for production.

Terraform State and remote state?

Terraform state is store/tracks the infrastructure data created by terraform

It stores resource IDs, IP addresses, attributes, and the mapping between code and real infrastructure.

Terraform Plan

terraform plan compares code + state + real infrastructure.

It determines what needs to be created, changed, or destroyed.

Remote State

By default, state is stored locally in terraform.tfstate.

In a team, I use a remote backend such as AWS S3.

Benefits

- *Centralized state for the team.*
- *Useful for CI/CD.*
- *Backup/versioning.*
- *State locking prevents concurrent changes.*

Security & Environment Management

- *Keep separate state for Dev, Staging, and Prod.*
- *Restrict access using IAM.*
- *Never store terraform.tfstate in Git because it can contain sensitive values.*

have you worked with Terraform modules?

Environment Module

We have separate environment configurations like:

- *Dev*
- *Staging*
- *Prod*

The environment contains environment-specific values such as variables, region, instance size, CIDR, etc.

It calls the Root Module.

Root Module

- *The root module is the main entry point for Terraform.*
- *It receives values from the environment module.*
- *The root module calls different Child Modules.*

Example:

Environment → Root Module → Child Modules

Child Modules

Child modules contain reusable infrastructure components.

For example:

```
modules/  
├─ vpc/  
├─ ec2/  
├─ eks/  
├─ rds/  
└─ iam/
```

Each module manages a specific infrastructure component.

Files inside a Child Module

main.tf	→ Resource definitions
variables.tf	→ Input variables
outputs.tf	→ Output values
locals.tf	→ Local values
data.tf	→ Data sources

Example:

```
modules/vpc/  
├─ main.tf  
├─ variables.tf  
├─ outputs.tf  
├─ locals.tf  
└─ data.tf
```

How do you protect the Terraform state file?

Remote backend + locking + encryption + access control.

"Remote backend with state locking (S3 + DynamoDB, or native locking on GCS/Azure), encryption at rest and in transit, versioning for rollback, and least-privilege IAM — state is never committed to git or run locally in production."

Terraform count vs for_each

count

- *count is a Terraform meta-argument.*
- *Used to create multiple instances of the same/similar resource.*
- *Works with a number or list.*
- *Resources are identified using index numbers:*

```
aws_instance.server[0]  
aws_instance.server[1]
```

- *Best when resources are almost identical.*
- *Main disadvantage: index shifting.*
 - *If you remove an item from the middle of a list, the indexes can change.*
 - *This can cause Terraform to destroy and recreate resources unnecessarily.*

for_each

- *for_each* is also a Terraform meta-argument.
- Used to create multiple resource instances using unique keys.
- Works with a map or set.
- Resources are identified using keys:

```
aws_instance.server["web"]
aws_instance.server["app"]
```

- Best when resources need different configurations.
- Provides stable resource identity.
- If you remove one key, Terraform generally removes only that specific resource.
- No index-shifting problem like `count`.

Terraform state is corrupted or deleted?

If the Terraform state is corrupted or deleted, Terraform can lose track of the infrastructure. I would not immediately apply changes. If using remote state like S3, I would restore the previous state version from backup/versioning. If no backup is available, I can use `terraform import` to bring existing resources back under Terraform management."

Explain a real scenario where terraform plan shows no change, but apply still modifies resources.

- **State drift:** Resources were changed manually outside Terraform (e.g., via cloud console or CLI).
 - Plan compares config vs. state file → no difference.
 - Apply refreshes state against real infra → detects drift → enforces config.
- **Computed attributes:** Some provider fields are marked as computed (e.g., timestamps, ARNs, auto-generated defaults).
 - Plan doesn't always show them.
 - Apply recalculates and updates them.
- **Provider defaults:** Cloud providers often inject defaults (tags, policies, flags).
 - Plan may not display these.
 - Apply enforces your explicit config, overwriting defaults.
- **Stale state file:** If the local state is outdated or corrupted, plan may miss differences.
 - Apply refreshes state before execution, catching mismatches.
- **Provider bugs/limitations:** Occasionally, providers fail to surface differences in plan but enforce them in apply.

Best practice: Run `terraform plan -refresh=true` or `terraform refresh` before trusting a "no changes" plan.

How provider version mismatches break production

- Different environments may use different provider versions.
- A newer provider version may introduce schema or behavior changes.
- Resource defaults may change between provider versions.
- The same Terraform code can behave differently with different provider versions.
- This can result in unexpected updates, replacements, or resource recreation.
- It can cause production configuration changes or downtime.

How to prevent it

- Pin the provider version in `required_providers`.
- Commit the `.terraform.lock.hcl` file to Git.
- Use the same Terraform and provider versions across environments.
- Don't upgrade providers directly in production.
- Test provider upgrades in a dev/test environment first.
- Run `terraform plan` and carefully review the changes before applying.

Resource vs Data Source?

Resource is used to create or manage AWS infrastructure, while a data source is used to read information about existing infrastructure.

what is the local?

Locals are similar to variables, but we use them to store reusable expressions and intermediate values inside Terraform. They help reduce code duplication and make the configuration easier to maintain.

it wil not override

What is Terraform output?

After creating a resource, we can use outputs to display important information about that resource.

We can also use that output value as an input for another module or resource.

How do you upgrade Terraform providers safely in production?

First, I check the provider changelog and compatibility. Then I test the new provider version in a lower environment like dev or staging. After testing, I update the provider version and run `terraform init -upgrade` and `terraform plan`. If the plan looks correct, I apply it to production with proper approval.

Steps to remember

Check compatibility
↓
Test in Dev/Staging
↓
Update provider version
↓
terraform init -upgrade
↓
terraform plan
↓
Review changes
↓
Production apply

Important points

- Pin the provider version.
- Commit `.terraform.lock.hcl`.
- Test before production.
- Always review terraform plan.
- Keep a backup of the Terraform state.
- Have a rollback/recovery plan.

How do you prevent accidental changes to production infrastructure?

Before applying changes, I review the terraform plan. I also use remote state with locking, code review, limited IAM permissions, and safeguards like `prevent_destroy` for critical resources.

Easy points to remember

1. Separate production state
2. Restrict production access
3. Review terraform plan
4. Code review before apply
5. Use remote state + locking
6. Limit IAM permissions
7. Use `prevent_destroy` for critical resources

How do you rename a Terraform resource without destroying it?

If I change the resource name directly in Terraform, Terraform may think it is a new resource and try to destroy the old one.

To avoid this, I use `terraform state mv` to rename the resource in the state file.

Example

Old:

```
resource "aws_instance" "app" {  
  # ...  
}
```

New:

```
resource "aws_instance" "web" {  
  # ...  
}
```

Then run:

```
terraform state mv aws_instance.app aws_instance.web
```

Now Terraform understands:

```
Old name: aws_instance.app  
    ↓  
Rename in state  
    ↓  
New name: aws_instance.web
```

The actual EC2 instance is not destroyed or recreated.

What is a null_resource in Terraform?

null_resource is a Terraform resource that does not create any real infrastructure.

It is mainly used to run provisioners or trigger actions based on changes.